

1. Integer values are implemented and manipulated at the hardware-level, allowing for fast operations. But the hardware does not supported unlimited integer values. For example, when using a 32-bit architecture, the integers are limited to the range -2,147,483,648 through 2,147,483,647. If you use a 64-bit architecture, this range is increased to the range -9,223,372,036,854,775,808 through 9,223,372,036,854,775,807. But what if we need more than 19 digits to represent an integer value?

In order to provide platform-independent integers and to support integers larger than 19 digits, Python implements its integer type in software. That means the storage and all of the operations that can be performed on the values are handled by executable instructions in the program and not by the hardware. Learning to implement integer values in software offers a good example of the need to provide efficient implementations. We define the Big Integer ADT below that can be used to store and manipulate integer values of any size, just like Python's built-in `int` type.

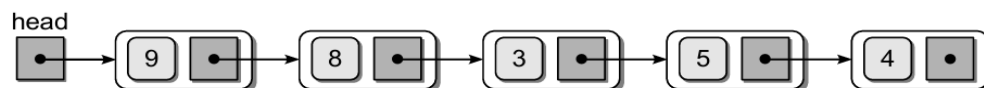
- `BigInteger(initValue = "0")`: Creates a new big integer that is initialized to the integer value specified by the given string.
- `toString()`: Returns a string representation of the big integer.
- `comparable(other)`: Compares this big integer to the `other` big integer to determine their logical ordering. This comparison can be done using any of the logical operators: `<`, `<=`, `>`, `>=`, `==`, `!=`.
- `arithmetic(rhsInt)`: Returns a new `BigInteger` object that is the result of performing one of the arithmetic operations on the `self` and `rhsInt` big integers. Any of the following operations can be performed:

+ - * // % **

- `bitwise-ops(rhsInt)`: Returns a new `BigInteger` object that is the result of performing one of the bitwise operators on the `self` and `rhsInt` big integers. Any of the following operations can be performed:

| & ^ << >>

- (a) Implement the Big Integer ADT using a singly linked list in which each digit of the integer value is stored in a separate node. The nodes should be ordered from the least-significant digit to the largest. For example, the linked list below represents the integer value 45,839:



- (b) Implement the Big Integer ADT using a Python list for storing the individual digits of an integer.

2. Modify your implementation of the Big Integer ADT from the previous question by adding the assignment combo operators that can be performed on the **self** and **rhsInt** big integers. Allow for any of the following operations to be performed:

+=	-=	*=	//=	%=	**=
<<=	>>=	=	&=	=	